

Esercizio 1 (9 punti) Scrivere una funzione `Anc(T,k1,k2)` che dato un albero binario di ricerca `T` nel quale tutte le chiavi sono distinte, e due chiavi `k1`, `k2` presenti in `T`, verifica se il nodo contenente `k1` è antenato del nodo contenente `k2`. Valutare la complessità della funzione.

Soluzione: È sufficiente osservare che, dato che le chiavi sono uniche e `k1`, `k2` sono certamente contenute in `T`, il nodo che contiene `k1` è antenato di quello che contiene `k2` se e solo se cercando `k2` a partire dalla radice di `T` incontro la chiave `k1`. Si assume che un nodo sia antenato di sé stesso.

La funzione può dunque essere realizzata come una semplice variante della ricerca negli alberi binari di ricerca.

```
Anc(T, k1, k2)
    x = T.root

    while (x.key <> k1) and (x.key <> k2)
        if (k2 < x.key)
            x = x.left
        else
            x = x.right

    return (x.key == k1)
```

Se l'albero ha altezza h , nel caso peggiore non trovo `k1` e la chiave `k2` è una foglia a profondità h , che quindi raggiungo in h iterazioni. Pertanto la complessità è $O(h)$.

Domanda B (7 punti) Scrivere una funzione `toTree(A)` che dato un array `A` organizzato a max-heap (dimensione `A.heapSize`), lo trasforma in un albero binario realizzato con strutture linked, ancora organizzato a max-heap e ritorna la radice di tale albero. Il nuovo albero è costituito da nodi `x` con i campi `x.p` (parent), `x.k` (chiave), `x.l` e `x.r` (figlio sinistro e figlio destro). Per allocare un nuovo nodo si assuma di avere a disposizione un costruttore `node()`. Valutare la complessità.

Soluzione: L'idea è semplicemente quella di visitare l'albero, memorizzato come array, in profondità, e riprodurlo nella struttura linked.

```
toTree(A)
    T.root = toTreeRec(A,1,nil)
    return T

// toTreeRec(A,i,x):
// dato un max-heap A ritorna la radice di un albero, copia "linked" del
// sottoalbero di A radicato in i, l'argomento x e' il nodo padre
toTreeRec(A,i,x)
    if i <= A.heapSize
        y = node()                // crea un nuovo nodo
        y.key = A[i]              // la chiave e' A[i]
        y.p = x                   // il parent e' x
        left = 2*i                // costruisce i sottoalberi sx e dx,
        right = 2*i+1             // che saranno figli sx e dx di y
        y.l = toTreeRec(A,left, y)
        y.r = toTreeRec(A,right,y)
    else
        return nil
```

Si tratta di una visita dell'albero e quindi la complessità è $\Theta(n)$ dove n è il numero di elementi del max-heap.

Esercizio 1 (10 punti) Dare la definizione di albero binario di ricerca. Scrivere una funzione `conta(T,a,b)` che dato un albero binario di ricerca `T` e due valori `a` e `b`, ritorna il numero di nodi `x` tali che `x.key` $\in [a, b]$. Mostrarne la correttezza e valutarne la complessità.

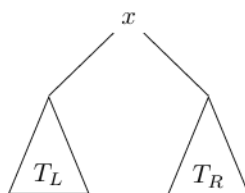
Soluzione: L'idea è quella di realizzare una funzione ricorsiva `contaRec(x,a,b)`, che dato un nodo `x`, restituisce il numero di nodi nel sottoalbero radicato in `x` con chiave nell'intervallo $[a, b]$. Quindi la si applica alla radice dell'albero.

```
conta(T,a,b)
    return contaRec(T,root, a,b)

contaRec(x, a, b)
    if x <> nil
        if b < x.key
            return contaRec(x.left)
        else if x.key < a
            return contaRec(x.right)
        else
            return 1 + contaRec(x.right) + contaRec(x.left)
```

La correttezza della funzione ricorsiva può essere dimostrata per induzione sull'altezza h del sottoalbero radicato in x

- ($h = 0$) In questo caso, $x = nil$, il sottoalbero è vuoto e quindi correttamente si ritorna 0.
- ($h > 0$) In questo caso l'albero ha la forma



Si distinguono tre casi:

- se $b < x.key$, sicuramente, per le proprietà degli ABR, tutti i nodi nel sottoalbero T_R hanno chiave maggiore di $x.key$ e quindi di b , e pertanto non sono nell'intervallo. Dunque si ritorna il numero di nodi in $[a, b]$ nel sottoalbero T_L , che per ipotesi induttiva è fornito da `contaRec(x.left)`.
- $x.key < a$ si ragiona in modo duale rispetto al caso precedente
- altrimenti, $x.key \in [a, b]$ e i nodi con chiave in $[a, b]$ possono trovarsi sia nel sottoalbero sinistro che nel sottoalbero destro. Si ritorna dunque $1 + \text{contaRec}(x.right) + \text{contaRec}(x.left)$, dove le chiamate `contaRec(x.right)` e `contaRec(x.left)` ritornano, per ipotesi induttiva, i nodi in $[a, b]$ nei sottoalberi T_R e T_L .

Nel caso peggiore, ad esempio quando tutti i nodi hanno chiave in $[a, b]$, la funzione esegue una visita completa dell'albero e quindi la complessità è $O(n)$ dove n è il numero dei nodi.

Domanda B (6 punti) Realizzare una funzione `SearchUnique(T,k)` che dato un albero binario di ricerca `T` (che si assume non vuoto), verifica se la chiave `k` è presente in un unico nodo, e, in caso affermativo restituisce il nodo, altrimenti (ovvero se la chiave non è presente oppure è presente in più nodi) restituisce `nil`. Si possono usare le funzioni standard degli alberi binari di ricerca. Valutarne la complessità.

Soluzione: L'idea è quella di cercare la chiave k con l'usuale ricerca dei BST. Se la chiave è presente, detto x il nodo trovato, per verificare se sia unica, si controlla che sottoalbero destro e sinistro non contengano ulteriori occorrenze della chiave. Questo si può fare con due ulteriori ricerche. La usuale funzione dei BST che cerca una chiave k nel sottoalbero radicato in x si indica con `Search(x,k)`.

```

SearchUnique(T,k)
  x = Search(T.root,k)

  if (x == nil) or (Search(x.left, k) <> nil) or (Search(x.right, k) <> nil)
    return x
  else
    return nil

```

L'algoritmo è corretto. L'osservazione fondamentale è che dato un nodo x di un BST, se la chiave k compare nel sottoalbero destro e in quello sinistro allora è anche la chiave della radice (infatti, per le proprietà dei BST, si avrebbe $k \leq x.key \leq k$). Ne segue che tutti i nodi con chiave k in T saranno nel sottoalbero del nodo x , ritornato dalla prima chiamata della funzione **Search**. Questo immediatamente implica la correttezza dell'algoritmo.

La complessità è quella di tre ricerche ciascuna con costo $O(h)$, dove h è l'altezza dell'albero, e quindi $O(h)$.

In alternativa, si può osservare che, se la chiave k dovesse comparire nel sottoalbero sinistro, sarebbe il massimo di questo sottoalbero e, dualmente, se comparisse nel secondo sottoalbero, sarebbe il minimo. Questo porta alla soluzione che segue

```

SearchUnique(T,k)
  x = Search(T.root,k)

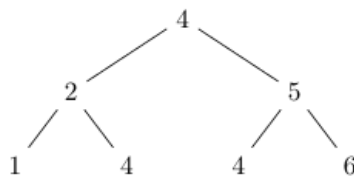
  if (x <> nil)
    y = Max(x.left)
    z = Min(x.right)

    if ((y <> nil) and (y.key == k)) or (z <> nil) and (z.key == k))
      return nil
    else
      return x
  else
    return nil

```

La complessità asintotica non cambia, ma questa funzione è più efficiente della precedente dato che **Min** e **Max**, diversamente a **Search** non eseguono confronti sulle chiavi.

Per concludere, è da osservare che l'assunzione che le chiavi duplicate siano solo nel sottoalbero destro (o in quello sinistro) o che siano adiacenti non è legittima. Ad esempio, il seguente è un BST valido:



Esercizio 1 (9 punti) Si consideri una variante degli alberi binari di ricerca nella quale i nodi x hanno un campo addizionale $x.min$, che rappresenta il minimo delle chiavi nel sottoalbero radicato in x . Realizzare la procedura $Insert(T, z)$ che inserisce un nodo z nell'albero e la rotazione a sinistra $Left(T, x)$ (assumendo che x e $x.right$ non siano nil). Valutarne la complessità.

Soluzione: La procedura di inserimento scende dalla radice fino ad una foglia dove il nodo viene inserito z . Dato che z sarà nel sottoalbero di ogni nodo attraversato durante la discesa, per ciascuno di questi nodi e soltanto per essi il campo min deve essere possibilmente aggiornato. Una piccola ottimizzazione: la verifica che $z.key < x.min$ ha senso farla solo se $z.key < x.key$ dato che $x.min \leq x.key$.

```
Insert(T, z)
    y = nil
    x = T.root

    while (x <> nil)
        y = x

        if (z.key < x.key)
            if z.key < x.min      /* il sottoalbero radicato in z contiene z */
                x.min = z.key    /* quindi si aggiorna x.min                */

            x = x.left
        else
            x = x.right

    z.p = y
    if (y <> nil)
        if (z.key < y.key)
            y.left = z
        else
            y.right = z

    z.min = z.key    /* il sottoalbero radicato in z contiene solo z */
```

La complessità rimane la stessa di *Insert* negli alberi binari di ricerca “standard”, ovvero $O(h)$, dove h è l'altezza dell'albero.

Anche la rotazione a sinistra si ottiene inserendo, in coda al codice standard, l'aggiornamento del campo min . Si osservi che gli unici nodi per i quali si modificano i nodi contenuti nel corrispondente sottoalbero sono x e $y = x.right$. I nodi che prima erano nel sottoalbero radicato in x poi sono in quello radicato in y , quindi i valori dei campi min e max per y dopo la rotazione sono quelli che prima erano in x . Per calcolare il nuovo valore del campo min per x si usa la conoscenza di tali valori per i figli destro e sinistro.

```
Left(T, z)
    y = x.right
    x.right = y.left
    x.right.p = x
    Transplant(T, x, y)
    y.left = x
    x.p = y

    y.min = x.min

    x.min = min(x.key, x.left.min, x.right.min)
```

La complessità resta costante $O(1)$.

Esercizio 1 (10 punti) Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x , il grado di bilanciamento del sottoalbero radicato in x , definito come $h_x / \log_2(n_x + 1)$ dove h_x e n_x indicano rispettivamente l'altezza e il numero di nodi del sottoalbero radicato in x (si intende che un albero costituito da un solo nodo abbia altezza 1).

Indicare quali campi occorre aggiungere ai nodi. Fornire lo pseudo-codice per la funzione `bal(x)` che restituisce il grado di bilanciamento del sottoalbero radicato in x e la procedura di inserimento di un nodo `insert(T,z)`. Valutare la complessità delle funzioni definite.

Soluzione: L'idea è quella di arricchire la struttura facendo in modo che ogni nodo x , oltre ai campi usuali, abbia due campi aggiuntivi: $x.h$ che contiene l'altezza del sottoalbero radicato in x e $x.size$ che contiene il numero dei nodi in tale sottoalbero.

A questo punto è immediato realizzare la funzione `bal(x)` di tempo costante

```
bal(x)
  if x <> nil
    return x.h/log_2(x.size)
  else
    error
```

Invece per l'inserimento, si modifica la procedura standard, che discende l'albero dalla radice fino alla posizione in cui inserire il nuovo nodo, facendo in modo che tutti i nodi attraversati e che quindi conterranno nel sottoalbero il nuovo nodo, abbiano il campo `size` incrementato di 1. Inoltre, una eventuale modifica dell'altezza va propagata verso l'alto. Questo accade quando il parent del nodo inserito era una foglia e quindi la sua altezza, prima dell'inserimento, era 1.

```
Insert(T,z)
  y = nil
  x = T.root

  while (x <> nil)
    y = x
    x.size++
    if (z.key < x.key)
      x = x.left
    else
      x = x.right

  // inizializza i campi del nodo z
  z.size = 1
  z.left = z.right = nil
  z.h = 1

  z.p = y
  if (y <> nil)      // se z non radice
    if (z.key < y.key)
      y.left = z
    else
      y.right = z

  // se y era una foglia, ovvero se l'altezza di y era 1, la sua
  // altezza e conseguentemente quella degli antenati va
  // aumentata di 1 risalendo i nodi attraversati nella discesa
  if y.h == 1
    while y <> nil
      y.h ++
      y = y.p
```

La complessità resta $O(h)$ dove h è l'altezza dell'albero.

Esercizio 1 (9 punti) Realizzare una funzione `avgTree(T)` che dato un albero binario `T` con chiavi numeriche, verifica se, per ogni nodo che abbia discendenti, la chiave del nodo è maggiore o uguale della media delle chiavi dei discendenti e ritorna conseguentemente un valore booleano (la radice dell'albero è `T.root` e ogni nodo `x` ha i campi `x.left`, `x.right` e `x.key`).

Soluzione: L'idea è quella di effettuare una visita dell'albero, in modo ricorsivo, raccogliendo per ogni sottoalbero le seguenti informazioni:

- se è soddisfatta la condizione sulla media richiesta
- la somma delle chiavi
- il numero dei nodi.

Quanto mi trovo su di un nodo, analizzo i sottoalberi sinistro e destro. Quindi, avendo a disposizione la somma delle chiavi ed il numero dei nodi per i due sottoalberi posso verificare la condizione sulla media per il nodo in esame. Lo pseudocodice della soluzione può essere il seguente:

```
avgTree(T)
    v, n, s = avgTreeRec(T.root)
    return v

# avgTreeRec(x):
# verifica se il sottoalbero radicato in x e' un avgTree e ritorna tre valori:
# - un booleano,
# - il numero dei discendenti
# - la somma delle loro chiavi

avgTreeRec(x)

if x = nil
    return true, 0, 0
else
    # ispeziono i sottoalberi sinistro e destro
    vl, nl, sl = avgTreeRec(x.left)
    vr, nr, sr = avgTreeRec(x.right)

    # se non ci sono discendenti (nl+nr =0) oppure la chiave del nodo in
    # esame e' maggiore o uguale della media delle chiavi dei discendenti
    # la condizione e' soddisfatta
    v = (nl+nr =0) or (x.key >= (sl+sr)/(nl+nr))

    # il numero di nodi dell'albero radicato in x e' dato dalla somma del
    # del numero di nodi nel sottoalbero dx e nel sottoalbero sx piu' uno
    # (il nodo x stesso)
    n = nl + nr + 1

    # la somma delle chiavi dell'albero radicato in x e' dato dalla somma del
    # delle chiavi nel sottoalbero dx e in quello sx piu' x.key
    s = sl + sr + x.key

    return v, n, s
```

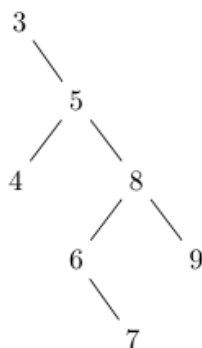
Si tratta di una visita e quindi la complessità è $\Theta(n)$.

Domanda B (7 punti) Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi:

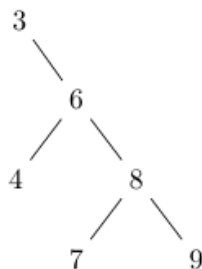
3, 5, 8, 6, 9, 7, 4

Dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

Soluzione: Per la definizione di albero binario di ricerca e la descrizione delle procedure di inserimento e cancellazione si consulti il libro. Sullo specifico esempio si ottiene, dopo gli inserimenti:



La cancellazione di 5 quindi produce:



Esercizio 1 (9 punti) Realizzare un arricchimento degli alberi binari di ricerca che permetta di ottenere per ogni nodo x , in tempo costante, la media dei valori memorizzati nel sottoalbero radicato in x .

Indicare quali campi occorre aggiungere ai nodi. Fornire il codice per la funzione `avg(x)` che restituisce la media delle chiavi nel sottoalbero radicato in x e la procedura di inserimento di un nodo `insert(T,z)`.

Soluzione: L'idea è quella di arricchire la struttura facendo in modo che ogni nodo x , oltre ai campi usuali, abbia il campo $x.sum$ che contiene la somma delle chiavi per i nodi del sottoalbero radicato in x e $x.size$ che contiene il numero dei nodi in tale sottoalbero.

A questo punto è immediato realizzare la funzione `avg(x)` di tempo costante

```
avg(x)
  if x <> nil
    return x.sum/x.size
  else
    error
```

Invece per l'inserimento, si modifica la procedura standard, che discende l'albero dalla radice fino alla posizione in cui inserire il nuovo nodo, facendo in modo che tutti i nodi attraversati e che quindi conterranno nel sottoalbero il nuovo nodo, abbiano i campi `sum` e `size` aggiornati.

```
Insert(T,z)
  y = nil
  x = T.root

  while (x <> nil)
    y = x
    x.size++
    x.sum = x.sum + z.key
    if (z.key < x.key)
      x = x.left
    else
      x = x.right

  z.p = y
  if (y <> nil)
    if (z.key < y.key)
      y.left = z
    else
      y.right = z

  z.sum = z.key
  z.size = 1
  z.left = z.right = nil
```

La complessità resta $O(h)$ dove h è l'altezza dell'albero.

Esercizio 1 (9 punti) Realizzare una funzione `append(T, x, z)` che dato un albero binario di ricerca T , un suo nodo *foglia* x ed un nuovo nodo z , verifica se è possibile inserire z come figlio (destro o sinistro) di x in T . In caso affermativo ritorna l'albero esteso, altrimenti ritorna `nil`. (Assumere che i nodi dell'albero abbiano gli usuali campi $x.left$, $x.right$, $x.p$, $x.key$ e la radice sia $T.root$.) Valutarne la complessità.

Soluzione: Affinchè z possa essere inserito come figlio destro o sinistro di x , occorre in primo luogo che la chiave di z sia $\leq$ delle chiavi degli antenati di z che hanno x e quindi z nel sottoalbero sinistro e $\geq$ delle chiavi degli antenati di x che hanno x nel sottoalbero destro.

Quindi risalendo tali antenati, si verifica che la condizione sia soddisfatta. Se questo accade e $z.key$ è $\leq x.key$ potrà essere inserito come figlio sinistro di x , altrimenti come figlio destro. Altrimenti z non può essere figlio di x e si ritorna `nil`.

Lo pseudocodice può essere il seguente:


```

append(T,x,z)

# verifica che z.key sia <= delle chiavi degli antenati di x
# per i quali x e' nel sottoalbero dx e
y = x

while (y.p <> nil) and
    ( (y == y.p.left   and  z.key <= y.p.key) or
      (y == y.p.right  and  z.key >= y.p.key) )
    y = y.p

# se siamo arrivati alla radice, significa che la condizione e' soddisfatta
if (y.p == nil)
    # e z viene inserito come figlio dx o sx
    z.p = x
    if (z.key <= x.key)
        x.left = z
    else
        x.right = z

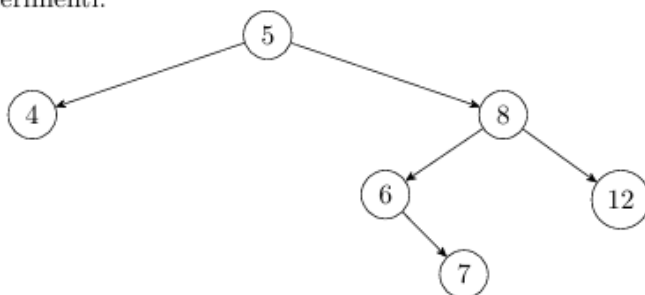
    return T
else
    # altrimenti si ritorna nil
    return nil

```

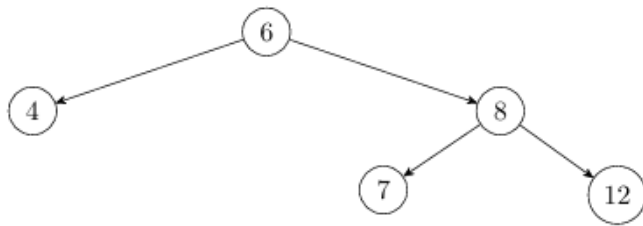
La complessità è chiaramente $O(h)$, dove h è l'altezza dell'albero. Infatti, il ciclo while effettua un numero di iterazioni pari al numero di antenati di x , e quindi limitato dall'altezza dell'albero. E ciascuna iterazione ha costo costante.

Domanda B (5 punti) Dare la definizione di albero binario di ricerca. Specificare l'albero ottenuto inserendo, con la procedura vista a lezione, a partire da un albero vuoto, i nodi aventi le seguenti chiavi: 5, 4, 8, 6, 12, 7. Si supponga che dall'albero così ottenuto si cancelli il nodo con chiave 5 e si indichi l'albero ottenuto. Sia per gli inserimenti che per la cancellazione, motivare sinteticamente il risultato ottenuto.

Soluzione: Per la definizione di albero binario di ricerca e la descrizione delle procedure di inserimento e cancellazione si consulti il libro. Sullo specifico esempio si ottiene, dopo gli inserimenti:



La cancellazione di 5 quindi produce:



Esercizio 1 (7 punti)

- Scrivere una funzione $\text{Rep}(T, k)$ che dato un albero di ricerca T e una chiave k ritorna il numero di occorrenze di k in T . Valutare la complessità dell'algoritmo definito.
- Se si assume che l'albero binario di ricerca non contenga chiavi ripetute cosa cambia? (Ovviamente in questo caso il numero di occorrenze è al più uno). Adattare la soluzione e discutere la relativa complessità.

Parte 1: ABR con Chiavi Ripetute

Strategia

In un ABR con duplicati, le chiavi uguali possono essere:

- Nel sottoalbero sinistro (convenzione: $\leq$ invece di $<$)
- Nel sottoalbero destro (convenzione: $\geq$ invece di $>$)
- Nel nodo corrente

Dobbiamo visitare entrambi i sottoalberi per contare tutte le occorrenze.

Pseudocodice

```
REP(T, k)
1. return REP_REC(T.root, k)

REP_REC(x, k)
// Conta occorrenze di k nel sottoalbero radicato in x
1. if x = nil
2.     return 0
3.
4. count = 0
5.
6. // Verifica nodo corrente
7. if x.key = k
8.     count = 1
9.
10. // Ricerca nei sottoalberi
11. left_count = REP_REC(x.left, k)
12. right_count = REP_REC(x.right, k)
```

```
13.  
14. return count + left_count + right_count
```

Versione Ottimizzata (con potatura)

Se conosciamo la **convenzione sui duplicati**, possiamo ottimizzare:

Convenzione A: Duplicati vanno a **sinistra** ($\leq$ a sinistra, $>$ a destra)

```
REP_LEFT_DUPLICATES(T, k)  
1. return REP_LEFT_REC(T.root, k)  
  
REP_LEFT_REC(x, k)  
1. if x = nil  
2.     return 0  
3.  
4. count = 0  
5. if x.key = k  
6.     count = 1  
7.  
8. if k < x.key  
9.     // k può essere solo a sinistra  
10.    return count + REP_LEFT_REC(x.left, k)  
11. else if k > x.key  
12.     // k può essere solo a destra  
13.    return REP_LEFT_REC(x.right, k)  
14. else // k = x.key  
15.     // Duplicati sono a sinistra per convenzione  
16.    return count + REP_LEFT_REC(x.left, k)
```

Convenzione B: Duplicati vanno a **destra** ($<$ a sinistra, $\geq$ a destra)

```
REP_RIGHT_DUPLICATES(T, k)  
1. return REP_RIGHT_REC(T.root, k)  
  
REP_RIGHT_REC(x, k)  
1. if x = nil  
2.     return 0  
3.  
4. count = 0  
5. if x.key = k  
6.     count = 1  
7.  
8. if k < x.key  
9.     return count + REP_RIGHT_REC(x.left, k)  
10. else if k > x.key  
11.     return REP_RIGHT_REC(x.right, k)  
12. else // k = x.key
```

```
13.    // Duplicati sono a destra per convenzione
14.    return count + REP_RIGHT_REC(x.right, k)
```

Analisi Complessità - Parte 1

Versione Base (senza ottimizzazioni)

Caso peggiore: Tutti i nodi hanno chiave k

- Visita tutto l'albero: $O(n)$

Caso migliore: k non è presente e albero è bilanciato

- Ma dobbiamo comunque visitare entrambi i sottoalberi di ogni nodo $\rightarrow O(n)$

Conclusione: La versione base è sempre $O(n)$ perché non possiamo escludere sottoalberi.

Versione Ottimizzata (con convenzione nota)

Caso peggiore: Tutti i nodi hanno chiave k

- Esempio: albero completamente sbilanciato con tutte chiavi $= k$
- Visita $O(n)$ nodi: $O(n)$

Caso medio: Poche occorrenze di k in albero bilanciato

- Trova primo k : $O(h) = O(\log n)$
- Conta duplicati nel sottoalbero appropriato: $O(d)$ dove d = numero duplicati
- **Totale:** $O(h + d) = O(\log n + d)$

Caso migliore: k non è presente, albero bilanciato

- Ricerca binaria: $O(h) = O(\log n)$
-

Parte 2: ABR Senza Chiavi Ripetute

Cosa Cambia?

Se l'ABR **non contiene duplicati**, allora:

- k può apparire **al massimo una volta**
- Possiamo fermarci appena troviamo k

- Possiamo usare la ricerca binaria classica

Soluzione Adattata

```
REP_NO_DUPLICATES(T, k)
1.  return REP_UNIQUE_REC(T.root, k)

REP_UNIQUE_REC(x, k)
// Ritorna 1 se k è presente, 0 altrimenti
1.  if x = nil
2.      return 0
3.
4.  if x.key = k
5.      return 1
6.
7.  if k < x.key
8.      return REP_UNIQUE_REC(x.left, k)
9.  else
10.     return REP_UNIQUE_REC(x.right, k)
```

Equivalente a: Ricerca binaria classica in ABR che ritorna booleano (0 o 1).

Analisi Complessità - Parte 2

Caso peggiore: k non presente in albero completamente sbilanciato

- Altezza $h = n$
- **Complessità: $O(n)$**

Caso medio: Albero bilanciato

- Altezza $h = \log n$
- **Complessità: $O(\log n)$**

Caso migliore: k è la radice

- **Complessità: $O(1)$**
-

Confronto tra le Due Versioni

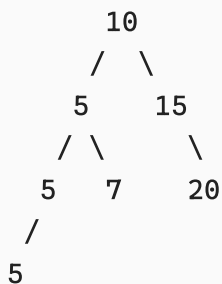
Aspetto	Con Duplicati	Senza Duplicati
Strategia	Visita ricorsiva completa o parziale	Ricerca binaria

Aspetto	Con Duplicati	Senza Duplicati
Caso peggiore	$O(n)$	$O(h) = O(n)$ sbilanciato
Caso medio (bilanciato)	$O(\log n + d)$ con potatura	$O(\log n)$
Caso migliore	$O(\log n)$ se k assente	$O(1)$ se k è radice
Complessità spaziale	$O(h)$ stack ricorsivo	$O(h)$ stack ricorsivo

Osservazione chiave: Senza duplicati, la complessità è **sempre $O(h)$** dove h è l'altezza dell'albero. Con duplicati, nel caso peggiore è **$O(n)$** se dobbiamo contare tutti i duplicati.

Esempi Concreti

Esempio 1: Con Duplicati (Convenzione: duplicati a sinistra)

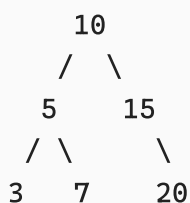


REP_LEFT_DUPLICATES(T, 5):

1. $x = 10$, $k=5 < 10 \rightarrow$ vai a sinistra
2. $x = 5$, $k=5 = 5 \rightarrow \text{count}=1$, vai a sinistra (duplicati a sx)
3. $x = 5$, $k=5 = 5 \rightarrow \text{count}=1$, vai a sinistra
4. $x = 5$, $k=5 = 5 \rightarrow \text{count}=1$, vai a sinistra
5. $x = \text{nil} \rightarrow \text{return } 0$

Somma: $1 + 1 + 1 + 0 = 3 \checkmark$

Esempio 2: Senza Duplicati



REP_NO_DUPLICATES(T, 5):

1. $x = 10$, $k=5 < 10 \rightarrow$ vai a sinistra
2. $x = 5$, $k=5 = 5 \rightarrow \text{return } 1$

Totale passi: $2 \cdot O(\log n)$

Dimostrazione di Correttezza

Con Duplicati (Versione Base)

Teorema: $\text{REP_REC}(x, k)$ restituisce il numero esatto di occorrenze di k nel sottoalbero radicato in x .

Dimostrazione per induzione sull'altezza h :

Base ($h=0$): x è foglia o nil

- Se $x = \text{nil}$: return 0 ✓
- Se x è foglia: conta solo $x.\text{key} = k$ se vero $\rightarrow$ corretto ✓

Passo ($h>0$):

- Per IP, $\text{REP_REC}(x.\text{left}, k)$ conta correttamente occorrenze a sinistra
- Per IP, $\text{REP_REC}(x.\text{right}, k)$ conta correttamente occorrenze a destra
- $\text{count} = 1$ se $x.\text{key} = k$, altrimenti 0
- Totale = $\text{count} + \text{left} + \text{right}$ = occorrenze totali ✓

Senza Duplicati

Teorema: $\text{REP_UNIQUE_REC}(x, k)$ restituisce 1 se k è presente, 0 altrimenti.

Dimostrazione: Equivalente alla dimostrazione di correttezza della ricerca binaria in ABR (standard). Per proprietà BST:

- Se $k < x.\text{key}$, k può essere solo a sinistra
- Se $k > x.\text{key}$, k può essere solo a destra
- Se $k = x.\text{key}$, trovato ✓

Versione Iterativa (Bonus - Senza Duplicati)

```
REP_ITERATIVE(T, k)
1.  x = T.root
2.  while x ≠ nil
3.      if x.key = k
4.          return 1
5.      if k < x.key
```

```
6.         x = x.left
7.     else
8.         x = x.right
9. return 0
```

Complessità: $O(h)$

Spazio: $O(1)$ - nessuna ricorsione

Conclusione

Risposta alle domande:

1. Con duplicati:

- Versione base: $O(n)$ sempre
- Versione ottimizzata (con convenzione): $O(h + d)$ dove d = numero duplicati

2. Senza duplicati:

- Complessità: $O(h)$ dove h è l'altezza
- In albero bilanciato: $O(\log n)$
- **Miglioramento sostanziale:** da $O(n)$ potenziale a $O(\log n)$ garantito in alberi bilanciati

Chiave del miglioramento: Senza duplicati possiamo usare la proprietà BST per escludere intere porzioni dell'albero durante la ricerca, riducendo da visita completa a ricerca binaria.

Si consideri una variante degli alberi binari di ricerca nella quale i nodi x hanno due campi aggiuntivi:

- un campo *pos* che permette di capire la posizione del nodo inserito
- un campo *delta* che permette di capire se dobbiamo ancora mappare le posizioni di altri nodi

Realizzare la procedura di $Insert(T, z)$ che inserisce un nodo z nell'albero e la procedura di stampa

$Print(T)$ che dato un albero lo stampa considerando i campi aggiunti.

```

insert(T, z)
  y = nil
  x = T.root
  while (x != nil)
    y = x
    if (z.key < x.key)
      x = x.left
    else
      x = x.right
  x.pos = x.pos + delta
  z.p = y
  if (y == nil)
    if (z.key < y.key)
      y.left = z
    else
      y.right = z
  z.pos = delta
  
```

```

Print(T)
  x = T.root
  if (x != nil) and (x.pos > 0)
    PrintPos(x.left)
    if (x.key > 0)
      Print(x.key)
    PrintPos(x.right)
  
```